Appendix A. Additional Details on ConGolog Semantics

The formal semantics of CONGOLOG is specified in terms of single-step transitions, using the following two predicates [6]:

- $Trans(\delta, s, \delta', s')$, which holds if one step of program δ in situation s may lead to situation s' with δ' remaining to be executed; and
- $Final(\delta, s)$, holding if program δ may legally terminate in situation s.

Note that since $Trans$ and $Final$ take programs (that include tests of formulas) as arguments, this requires encoding formulas and programs as terms.

Predicate $Do(\delta, s, s')$ means that program δ, when executed starting in situation s, has as a legal terminating situation s', and is defined as

$$Do(\delta, s, s') \doteq \exists \delta'. Trans^*(\delta, s, \delta', s') \wedge Final(\delta', s')$$

where $Trans^*$ denotes the reflexive transitive closure of $Trans$.
Assuming free variables to be universally quantified, the predicate $Trans$ is characterized by the following set of axioms:

Empty program:

$$Trans(nil, s, \delta', s') \equiv False$$

Primitive action:

$$Trans(\alpha, s, \delta', s') \equiv Poss(\alpha[s], s) \wedge \delta' = nil \wedge s' = do(\alpha[s], s)$$

Test action:

$$Trans(\varphi?, s, \delta', s') \equiv \varphi[s] \wedge \delta' = nil \wedge s' = s$$

Sequence:

$$\begin{aligned} Trans(\delta_1; \delta_2, s, \delta', s') \equiv & (\exists \delta_1'.\ \delta' = \delta_1'; \delta_2 \wedge Trans(\delta_1, s, \delta_1', s')) \vee \\ & (Final(\delta_1, s) \wedge Trans(\delta_2, s, \delta', s')) \end{aligned}$$

Conditional:

$$\begin{aligned} Trans(\textbf{if } \varphi \textbf{ then } \delta_1 \textbf{ else } \delta_2, s, \delta', s') \equiv & (\varphi[s] \wedge Trans(\delta_1, s, \delta', s')) \vee \\ & (\neg\varphi[s] \wedge Trans(\delta_2, s, \delta', s')) \end{aligned}$$

While loop:

$$
\begin{aligned}
Trans(\textbf{while } \varphi \textbf{ do } \delta, s, \delta', s') \equiv\ & \exists \delta''.\varphi[s] \wedge Trans(\delta, s, \delta'', s') \wedge \\
& \delta' = \delta''; (\textbf{while } \varphi \textbf{ do } \delta)
\end{aligned}
$$

Nondeterministic choice:

$$Trans(\delta_1 \mid \delta_2, s, \delta', s') \equiv Trans(\delta_1, s, \delta', s') \vee Trans(\delta_2, s, \delta', s')$$

Nondeterministic choice of argument:

$$Trans(\pi x.\delta, s, \delta', s') \equiv \exists x.\ Trans(\delta, s, \delta', s')$$

Iteration:

$$Trans(\delta^*, s, \delta', s') \equiv \exists \delta''.Trans(\delta, s, \delta'', s') \wedge \delta' = \delta''; \delta^*$$

Concurrency:

$$
\begin{aligned}
Trans(\delta_1 \parallel \delta_2, s, \delta', s') \equiv\ & (\exists \delta_1'.\ Trans(\delta_1, s, \delta_1', s') \wedge \delta' = \delta_1' \parallel \delta_2) \vee \\
& (\exists \delta_2'.\ Trans(\delta_2, s, \delta_2', s') \wedge \delta' = \delta_1 \parallel \delta_2')
\end{aligned}
$$

Prioritized concurrency:

$$
\begin{aligned}
Trans(\delta_1 \rangle\rangle \delta_2, s, \delta', s') \equiv\ & (\exists \delta_1'.\ Trans(\delta_1, s, \delta_1', s') \wedge \delta' = \delta_1' \rangle\rangle \delta_2) \vee \\
& (\exists \delta_2'.\ Trans(\delta_2, s, \delta_2', s') \wedge \delta' = \delta_1 \rangle\rangle \delta_2' \wedge \\
& \neg \exists \delta_1'', s''.Trans(\delta_1, s, \delta_1'', s''))
\end{aligned}
$$

Concurrent iteration:

$$Trans(\delta^{\parallel}, s, \delta', s') \equiv \exists \delta''.Trans(\delta, s, \delta'', s') \wedge \delta' = \delta'' \parallel \delta^{\parallel}$$

The predicate $Final$ is characterized by the following set of axioms:

Empty program:

$$Final(nil, s) \equiv True$$

Primitive action:

$$Final(\alpha, s) \equiv False$$

Test action:

$$Final(\varphi?, s) \equiv False$$

Sequence:

$$Final(\delta_1;\delta_2,s) \equiv Final(\delta_1,s) \wedge Final(\delta_2,s)$$

Conditional:

$$\begin{aligned} Final(\textbf{if } \varphi \textbf{ then } \delta_1 \textbf{ else } \delta_2, s) \equiv {} & (\varphi[s] \wedge Final(\delta_1,s)) \vee \\ & (\neg\varphi[s] \wedge Final(\delta_2,s)) \end{aligned}$$

While loop:

$$Final(\textbf{while } \varphi \textbf{ do } \delta, s) \equiv (\neg\varphi[s]) \vee Final(\delta,s)$$

Nondeterministic choice:

$$Final(\delta_1 \mid \delta_2, s) \equiv Final(\delta_1,s) \vee Final(\delta_2,s)$$

Nondeterministic choice of argument:

$$Final(\pi x.\delta, s) \equiv \exists x.\ Final(\delta,s)$$

Iteration:

$$Final(\delta^*, s) \equiv True$$

Concurrency:

$$Final(\delta_1 \parallel \delta_2, s) \equiv Final(\delta_1,s) \wedge Final(\delta_2,s)$$

Prioritized concurrency:

$$Final(\delta_1 \rangle\rangle \delta_2, s) \equiv Final(\delta_1,s) \wedge Final(\delta_2,s)$$

Concurrent iteration:

$$Final(\delta^{\parallel}, s) \equiv True$$

Appendix B. Case Study Complete Formalization

Here, we report the complete formalization of the case study presented in Section 5, which focuses on the job application business process model.

Appendix B.1. Basic Action Theory

The underlying basic action theory used in the case study is as follows.

The set of (relational) fluents is:

- $Active(id, s)$: It indicates whether the application id is being processed by the applicant pool in situation s.
- $Processed(id, s)$: It reports whether the application id is under analysis by the company pool in situation s.
- $Documents\_ok(id, s)$: It represents the result of the validity check on the application performed by the company pool in situation s and drives accordingly the execution flow towards the corresponding exclusive gateway.
- $Interview(id, s)$: It reports the result of the check determining whether the candidate who submitted the application is suitable for an interview in situation s, as assessed by the company pool, and accordingly directs the execution flow toward the corresponding exclusive gateway.
- $Job\_offer(id, s)$: It indicates the results of the execution of the interview for assessing the candidate in situation s and drives accordingly the sequence flow after the exclusive gateway.
- $Approval(id, s)$: It represents the approval of the formalized job offer for the applicant by the company in situation s, ruling the execution flow into the corresponding exclusive gateway.
- $Running(\alpha_s, s)$: It represents the initiation of a task and holds if the starting action is ongoing and the completing action is not yet performed.
- $Done(\alpha_e, s)$: It represents the completion of a task.
- $Application(id, s)$: It reports whether an application data object was created for the process instance id and exists in situation s.
- $Contract(id, s)$: It reports whether a contract data object is present for the process instance id in situation s.
- $Signed\_contract(id, s)$: It reports if the contract was signed for the process instance id.
- $Letter\_of\_refusal(id, s)$: It represents the presence of a letter of refusal data object for the process instance id in situation s.
- $Pool(p, s)$: It reports the considered pool (either the applicant or the company).
- $Waiting(id, pool, s)$: It tells whether a specified pool $pool$ is waiting for that process instance id to be processed in situation s.

- *Servers_stopped*(s): It indicates whether the servers associated with the pools are active in situation s or were stopped during the execution and, consequently, are unable to accept and process new requests.

In turn, the set of actions is:

$$\begin{aligned}\mathcal{A} = \{&\textsc{prepare\_application\_start}(id), \textsc{application\_sent}(id),\\ &\textsc{check\_validity\_start}(id), \textsc{organize\_documents\_start}(id),\\ &\textsc{verify\_prerequisites\_start}(id), \textsc{assign\_contact\_partner\_start}(id),\\ &\textsc{check\_application\_for\_interview\_start}(id),\\ &\textsc{plan\_interview\_start}(id), \textsc{execute\_interview\_start}(id),\\ &\textsc{obtain\_approval\_start}(id), \textsc{validate\_job\_offer\_start}(id),\\ &\textsc{produce\_contract\_start}(id), \textsc{contract\_sent}(id),\\ &\textsc{sign\_contract\_start}(id), \textsc{signed\_contract\_sent}(id),\\ &\textsc{store\_signed\_contract\_start}(id),\\ &\textsc{communicate\_recruitment\_start}(id),\\ &\textsc{application\_analysed}(id), \textsc{produce\_letter\_of\_refusal\_start}(id),\\ &\textsc{letter\_of\_refusal\_sent}(id), \textsc{application\_finalised}(id),\\ &\textsc{confirm\_withdrawal\_start}(id), \textsc{withdrawal\_completed}(id),\\ &\textsc{withdrawal\_sent}(id), \textsc{process\_withdrawal\_start}(id),\\ &\textsc{withdrawal\_handled}(id), \textsc{acquire}(id, pool), \textsc{job\_needed}(id),\\ &\textsc{prepare\_application\_end}(id), \textsc{check\_validity\_end}(id, res),\\ &\textsc{organize\_documents\_end}(id), \textsc{verify\_prerequisites\_end}(id),\\ &\textsc{assign\_contact\_partner\_end}(id), \textsc{plan\_interview\_end}(id),\\ &\textsc{check\_application\_for\_interview\_end}(id, res),\\ &\textsc{execute\_interview\_end}(id, res), \textsc{obtain\_approval\_end}(id),\\ &\textsc{produce\_contract\_end}(id), \textsc{validate\_job\_offer\_end}(id, res),\\ &\textsc{sign\_contract\_end}(id), \textsc{store\_signed\_contract\_end}(id),\\ &\textsc{communicate\_recruitment\_end}(id),\\ &\textsc{produce\_letter\_of\_refusal\_end}(id),\\ &\textsc{withdrawal\_by\_applicant}(id), \textsc{confirm\_withdrawal\_end}(id),\\ &\textsc{process\_withdrawal\_end}(id), \textsc{shut\_down}\}\end{aligned}$$

The following abbreviation defines which of the actions are *exogenous*:

$$
\begin{aligned}
Exog(a) \equiv\ & a = \textsc{job\_needed}(id)\ \vee \\
& a = \textsc{prepare\_application\_end}(id)\ \vee \\
& a = \textsc{check\_validity\_end}(id, res)\ \vee \\
& a = \textsc{organize\_documents\_end}(id)\ \vee \\
& a = \textsc{verify\_prerequisites\_end}(id)\ \vee \\
& a = \textsc{assign\_contact\_partner\_end}(id)\ \vee \\
& a = \textsc{check\_application\_for\_interview\_end}(id, res)\ \vee \\
& a = \textsc{plan\_interview\_end}(id)\ \vee \\
& a = \textsc{execute\_interview\_end}(id, res)\ \vee \\
& a = \textsc{obtain\_approval\_end}(id)\ \vee \\
& a = \textsc{produce\_contract\_end}(id)\ \vee \\
& a = \textsc{validate\_job\_offer\_end}(id, res)\ \vee \\
& a = \textsc{sign\_contract\_end}(id)\ \vee \\
& a = \textsc{communicate\_recruitment\_end}(id)\ \vee \\
& a = \textsc{store\_signed\_contract\_end}(id)\ \vee \\
& a = \textsc{produce\_letter\_of\_refusal\_end}(id)\ \vee \\
& a = \textsc{withdrawal\_by\_applicant}(id)\ \vee \\
& a = \textsc{confirm\_withdrawal\_end}(id)\ \vee \\
& a = \textsc{process\_withdrawal\_end}(id)\ \vee \\
& a = \textsc{shut\_down}
\end{aligned}
$$

Appendix B.1.1. Action Precondition Axioms

$$Poss(\textsc{prepare\_application\_start}(id), s) \equiv Active(id, s)$$
$$Poss(\textsc{application\_sent}(id), s) \equiv Application(id, s) \wedge Done(\textsc{prepare\_application\_end}(id), s)$$
$$Poss(\textsc{check\_validity\_start}(id), s) \equiv Application(id, s) \wedge Done(\textsc{application\_sent}(id), s)$$
$$Poss(\textsc{organize\_documents\_start}(id), s) \equiv Documents\_ok(id, s)$$
$$Poss(\textsc{assign\_contact\_partner\_start}(id), s) \equiv Done(\textsc{organize\_documents\_end}(id), s)$$
$$Poss(\textsc{verify\_prerequisites\_start}(id), s) \equiv Done(\textsc{organize\_documents\_end}(id), s)$$
$$Poss(\textsc{check\_application\_for\_interview\_start}(id), s) \equiv Application(id, s) \wedge Done(\textsc{assign\_contact\_partner\_end}(id), s) \wedge Done(\textsc{verify\_prerequisites\_end}(id), s)$$
$$Poss(\textsc{plan\_interview\_start}(id), s) \equiv Interview(id, s)$$
$$Poss(\textsc{execute\_interview\_start}(id), s) \equiv Done(\textsc{plan\_interview\_end}(id), s)$$
$$Poss(\textsc{obtain\_approval\_start}(id), s) \equiv Job\_offer(id, s)$$
$$Poss(\textsc{validate\_job\_offer\_start}(id), s) \equiv Done(\textsc{obtain\_approval\_end}(id), s)$$
$$Poss(\textsc{produce\_contract\_start}(id), s) \equiv Approval(id, s)$$
$$Poss(\textsc{contract\_sent}(id), s) \equiv Contract(id, s) \wedge Done(\textsc{produce\_contract\_end}(id), s))$$
$$Poss(\textsc{sign\_contract\_start}(id), s) \equiv Contract(id, s) \wedge Done(\textsc{contract\_sent}(id), s)$$
$$Poss(\textsc{signed\_contract\_sent}(id), s) \equiv Signed\_contract(id, s) \wedge Done(\textsc{sign\_contract\_end}(id), s)$$
$$Poss(\textsc{communicate\_recruitment\_start}(id), s) \equiv Signed\_contract(id, s) \wedge Done(\textsc{signed\_contract\_sent}(id), s)$$
$$Poss(\textsc{application\_finalised}(id), s) \equiv Done(\textsc{communicate\_recruitment\_end}(id), s) \vee Done(\textsc{letter\_of\_refusal\_sent}(id), s)$$
$$Poss(\textsc{store\_signed\_contract\_start}(id), s) \equiv Signed\_contract(id, s) \wedge Done(\textsc{signed\_contract\_sent}(id), s)$$

$$\begin{aligned}
&Poss(\textsc{application_analysed}(id), s) \equiv Done(\textsc{application_finalised}(id))\wedge\\
&\qquad\neg Done(\textsc{withdrawal_by_applicant}(id))\wedge\\
&\qquad(Done(\textsc{store_signed_contract_end}(id), s)\vee\\
&\qquad Done(\textsc{letter_of_refusal_sent}(id), s))\\
&Poss(\textsc{produce_letter_of_refusal_start}(id), s) \equiv\\
&\qquad\neg(Documents_ok(id, s) \wedge Interview(id, s)\wedge\\
&\qquad Job_offer(id, s) \wedge Approval(id, s))\\
&Poss(\textsc{letter_of_refusal_sent}(id), s) \equiv Letter_of_refusal(id, s)\wedge\\
&\qquad Done(\textsc{produce_letter_of_refusal_end}(id), s)\\
&Poss(\textsc{confirm_withdrawal_start}(id), s) \equiv\\
&\qquad Done(\textsc{withdrawal_by_applicant}(id), s)\\
&Poss(\textsc{withdrawal_sent}(id), s) \equiv\\
&\qquad Done(\textsc{confirm_withdrawal_end}(id), s)\\
&Poss(\textsc{withdrawal_completed}(id), s) \equiv Done(\textsc{withdrawal_sent}(id), s)\\
&Poss(\textsc{process_withdrawal_start}(id), s) \equiv\\
&\qquad Done(\textsc{withdrawal_sent}(id), s)\\
&Poss(\textsc{withdrawal_handled}(id), s) \equiv\\
&\qquad Done(\textsc{process_withdrawal_end}(id), s)\\
&Poss(\textsc{acquire}(id, pool), s) \equiv Waiting(id, pool, s) \wedge Pool(pool, s)\\
&Poss(\textsc{job_needed}(id), s) \equiv \neg Active(id, s)\\
&Poss(\textsc{prepare_application_end}(id), s) \equiv\\
&\qquad Running(\textsc{prepare_application_start}(id), s)\\
&Poss(\textsc{check_validity_end}(id, res), s) \equiv Application(id, s)\wedge\\
&\qquad Running(\textsc{check_validity_start}(id), s)\\
&Poss(\textsc{organize_documents_end}(id), s) \equiv\\
&\qquad Running(\textsc{organize_documents_start}(id), s)\\
&Poss(\textsc{verify_prerequisites_end}(id), s) \equiv\\
&\qquad Running(\textsc{verify_prerequisites_start}(id), s)\\
&Poss(\textsc{assign_contact_partner_end}(id), s) \equiv\\
&\qquad Running(\textsc{assign_contact_partner_start}(id), s)\\
&Poss(\textsc{check_application_for_interview_end}(id, res), s) \equiv\\
&\quad Application(id, s)\wedge\\
&\quad Running(\textsc{check_application_for_interview_start}(id), s)\\
&Poss(\textsc{plan_interview_end}(id), s) \equiv\\
&\qquad Running(\textsc{plan_interview_start}(id), s)\\
&Poss(\textsc{execute_interview_end}(id, res), s) \equiv\\
&\qquad Running(\textsc{execute_interview_start}(id), s)
\end{aligned}$$

$$
\begin{aligned}
&Poss(\textsc{obtain\_approval\_end}(id), s) \equiv \\
&\qquad\qquad Running(\textsc{obtain\_approval\_start}(id), s) \\
&Poss(\textsc{produce\_contract\_end}(id), s) \equiv \\
&\qquad\qquad Running(\textsc{produce\_contract\_start}(id), s) \\
&Poss(\textsc{validate\_job\_offer\_end}(id, res), s) \equiv \\
&\qquad\qquad Running(\textsc{validate\_job\_offer\_start}(id), s) \\
&Poss(\textsc{communicate\_recruitment\_end}(id), s) \equiv \\
&\qquad\qquad Running(\textsc{communicate\_recruitment\_start}(id), s) \\
&Poss(\textsc{sign\_contract\_end}(id), s) \equiv Contract(id, s) \wedge \\
&\qquad\qquad Running(\textsc{sign\_contract\_start}(id), s) \\
&Poss(\textsc{store\_signed\_contract\_end}(id), s) \equiv Signed\_contract(id, s) \wedge \\
&\qquad\qquad Running(\textsc{store\_signed\_contract\_start}(id), s) \\
&Poss(\textsc{produce\_letter\_of\_refusal\_end}(id), s) \equiv \\
&\qquad\qquad Running(\textsc{produce\_letter\_of\_refusal\_start}(id), s) \\
&Poss(\textsc{withdrawal\_by\_applicant}(id), s) \equiv Active(id, s) \wedge \neg Processed(id, s) \wedge \\
&\qquad\qquad \neg Done(\textsc{withdrawal\_by\_applicant}(id), s) \wedge \\
&\qquad\qquad \neg Done(\textsc{application\_analysed}(id), s) \wedge \\
&\qquad\qquad \neg Done(\textsc{application\_finalised}(id), s) \wedge \\
&\qquad\qquad \neg Waiting(id, p, s) \ \wedge \ Pool(p, s) \\
&Poss(\textsc{confirm\_withdrawal\_end}(id), s) \equiv \\
&\qquad\qquad Running(\textsc{confirm\_withdrawal\_start}(id), s) \\
&Poss(\textsc{process\_withdrawal\_end}(id), s) \equiv \\
&\qquad\qquad Running(\textsc{process\_withdrawal\_start}(id), s) \\
&Poss(\textsc{shut\_down}, s) \equiv \neg Servers\_stopped(s) \ \wedge \neg(\exists id. Active(id, s))
\end{aligned}
$$

Appendix B.1.2. Effect Axioms

$$a = \textsc{job\_needed}(id) \supset Waiting(id, applicant, do(a,s))$$
$$a = \textsc{job\_needed}(id) \supset Active(id, do(a,s))$$
$$a = \textsc{prepare\_application\_start}(id) \supset$$
$$\qquad Running(\textsc{prepare\_application\_start}(id), do(a,s))$$
$$a = \textsc{prepare\_application\_end}(id) \supset$$
$$\qquad Done(\textsc{prepare\_application\_end}(id), do(a,s))$$
$$a = \textsc{prepare\_application\_end}(id) \supset Application(id, do(a,s))$$
$$a = \textsc{application\_sent}(id) \supset Waiting(id, company, do(a,s))$$
$$a = \textsc{application\_sent}(id) \supset Done(\textsc{application\_sent}(id), do(a,s))$$
$$a = \textsc{check\_validity\_start}(id) \supset$$
$$\qquad Running(\textsc{check\_validity\_start}(id), do(a,s))$$
$$a = \textsc{check\_validity\_end}(id, res) \supset$$
$$\qquad Done(\textsc{check\_validity\_end}(id), do(a,s))$$
$$a = \textsc{check\_validity\_end}(id, res) \wedge res = true \supset$$
$$\qquad Documents\_ok(id, do(a,s))$$
$$a = \textsc{check\_validity\_end}(id, res) \wedge res = false \supset$$
$$\qquad \neg Documents\_ok(id, do(a,s))$$
$$a = \textsc{organize\_documents\_start}(id) \supset$$
$$\qquad Running(\textsc{organize\_documents\_start}(id, do(a,s))$$
$$a = \textsc{organize\_documents\_end}(id) \supset$$
$$\qquad Done(\textsc{organize\_documents\_end}(id), do(a,s))$$
$$a = \textsc{verify\_prerequisites\_start}(id) \supset$$
$$\qquad Running(\textsc{verify\_prerequisites\_start}(id, do(a,s))$$
$$a = \textsc{verify\_prerequisites\_end}(id) \supset$$
$$\qquad Done(\textsc{verify\_prerequisites\_end}(id), do(a,s))$$
$$a = \textsc{assign\_contact\_partner\_start}(id) \supset$$
$$\qquad \textsc{Running}(\textsc{assign\_contact\_partner\_start}(id), do(a,s))$$
$$a = \textsc{assign\_contact\_partner\_end}(id) \supset$$
$$\qquad Done(\textsc{assign\_contact\_partner\_end}(id), do(a,s))$$

$$a = \text{CHECK\_APPLICATION\_FOR\_INTERVIEW\_START}(id) \supset$$
$$Running(\text{CHECK\_APPLICATION\_FOR\_INTERVIEW\_START}(id), do(a, s))$$
$$a = \text{CHECK\_APPLICATION\_FOR\_INTERVIEW\_END}(id, res) \supset$$
$$Done(\text{CHECK\_APPLICATION\_FOR\_INTERVIEW\_END}(id), do(a, s))$$
$$a = \text{CHECK\_APPLICATION\_FOR\_INTERVIEW\_END}(id, res) \wedge res = true \supset$$
$$Interview(id, do(a, s))$$
$$a = \text{CHECK\_APPLICATION\_FOR\_INTERVIEW\_END}(id, res) \wedge res = false \supset$$
$$\neg Interview(id, do(a, s))$$
$$a = \text{PLAN\_INTERVIEW\_START}(id) \supset$$
$$Running(\text{PLAN\_INTERVIEW\_START}(id), do(a, s))$$
$$a = \text{PLAN\_INTERVIEW\_END}(id) \supset$$
$$Done(\text{PLAN\_INTERVIEW\_END}(id), do(a, s))$$
$$a = \text{EXECUTE\_INTERVIEW\_START}(id) \supset$$
$$\text{RUNNING}(\text{EXECUTE\_INTERVIEW\_START}(\text{ID}), do(a, s))$$
$$a = \text{EXECUTE\_INTERVIEW\_END}(id, res) \supset$$
$$Done(\text{EXECUTE\_INTERVIEW\_END}(id), do(a, s))$$
$$a = \text{EXECUTE\_INTERVIEW\_END}(id, res) \wedge res = true \supset$$
$$Job\_offer(id, do(a, s))$$
$$a = \text{EXECUTE\_INTERVIEW\_END}(id, res) \wedge res = false \supset$$
$$\neg Job\_offer(id, do(a, s))$$
$$a = \text{OBTAIN\_APPROVAL\_START}(id) \supset$$
$$Running(\text{OBTAIN\_APPROVAL\_START}(id), do(a, s))$$
$$a = \text{OBTAIN\_APPROVAL\_END}(id) \supset$$
$$Done(\text{OBTAIN\_APPROVAL\_END}(id), do(a, s))$$
$$a = \text{VALIDATE\_JOB\_OFFER\_START}(id) \supset$$
$$Running(\text{VALIDATE\_JOB\_OFFER\_START}(id), do(a, s))$$
$$a = \text{VALIDATE\_JOB\_OFFER\_END}(id, res) \supset$$
$$Done(\text{VALIDATE\_JOB\_OFFER\_END}(id), do(a, s))$$
$$a = \text{VALIDATE\_JOB\_OFFER\_END}(id, res) \wedge res = true \supset$$
$$Approval(id, do(a, s))$$

$a = \text{PRODUCE\_CONTRACT\_START}(id) \supset$
$\quad Running(\text{PRODUCE\_CONTRACT\_START}(id), do(a, s))$
$a = \text{PRODUCE\_CONTRACT\_END}(id) \supset$
$\quad Done(\text{PRODUCE\_CONTRACT\_END}(id), do(a, s))$
$a = \text{PRODUCE\_CONTRACT\_END}(id) \supset Contract(id, do(a, s))$
$a = \text{CONTRACT\_SENT}(id) \supset Done(\text{CONTRACT\_SENT}(id), do(a, s))$
$a = \text{SIGN\_CONTRACT\_START}(id) \supset$
$\quad Running(\text{SIGN\_CONTRACT\_START}(id), do(a, s))$
$a = \text{SIGN\_CONTRACT\_END}(id) \supset Done(\text{SIGN\_CONTRACT\_END}(id), do(a, s))$
$a = \text{SIGN\_CONTRACT\_END}(id) \supset Signed\_contract(id, do(a, s))$
$a = \text{SIGNED\_CONTRACT\_SENT}(id) \supset$
$\quad Done(\text{SIGNED\_CONTRACT\_SENT}(id), do(a, s))$
$a = \text{APPLICATION\_FINALISED}(id) \supset \neg Active(id, do(a, s))$
$a = \text{APPLICATION\_FINALISED}(id) \supset \neg Waiting(id, applicant, do(a, s))$
$a = \text{APPLICATION\_FINALISED}(id) \supset$
$\quad Done(\text{APPLICATION\_FINALISED}(id), do(a, s))$
$a = \text{STORE\_SIGNED\_CONTRACT\_START}(id) \supset$
$\quad \text{RUNNING}(\text{STORE\_SIGNED\_CONTRACT\_START}(id), do(a, s))$
$a = \text{STORE\_SIGNED\_CONTRACT\_END}(id) \supset$
$\quad Done(\text{STORE\_SIGNED\_CONTRACT\_END}(id, do(a, s))$
$a = \text{APPLICATION\_ANALYSED}(id) \supset Processed(id, do(a, s))$
$a = \text{APPLICATION\_ANALYSED}(id) \supset \neg Waiting(id, company, do(a, s))$
$a = \text{COMMUNICATE\_RECRUITMENT\_START}(id) \supset$
$\quad Running(\text{COMMUNICATE\_RECRUITMENT\_START}(id), do(a, s))$
$a = \text{COMMUNICATE\_RECRUITMENT\_END}(id) \supset$
$\quad Done(\text{COMMUNICATE\_RECRUITMENT\_END}(id), do(a, s))$
$a = \text{PRODUCE\_LETTER\_OF\_REFUSAL\_START}(id) \supset$
$\quad Running(\text{PRODUCE\_LETTER\_OF\_REFUSAL\_START}(id), do(a, s))$
$a = \text{PRODUCE\_LETTER\_OF\_REFUSAL\_END}(id) \supset$
$\quad Done(\text{PRODUCE\_LETTER\_OF\_REFUSAL\_END}(id), do(a, s))$

$a = \textsc{produce\_letter\_of\_refusal\_end}(id) \supset$
$\quad Letter\_of\_refusal(id, do(a, s))$
$a = \textsc{letter\_of\_refusal\_sent}(id) \supset$
$\quad Done(\textsc{letter\_of\_refusal\_sent}(id), do(a, s))$
$a = \textsc{withdrawal\_by\_applicant}(id) \wedge$
$\quad \neg Done(\textsc{application\_analysed}(id) \supset$
$\quad Done(\textsc{withdrawal\_by\_applicant}(id), do(a, s))$
$a = \textsc{confirm\_withdrawal\_start}(id) \supset$
$\quad Running(\textsc{confirm\_withdrawal\_start}(id), do(a, s))$
$a = \textsc{confirm\_withdrawal\_end}(id) \supset$
$\quad Done(\textsc{confirm\_withdrawal\_end}(id), do(a, s))$
$a = \textsc{withdrawal\_sent}(id) \supset$
$\quad Done(\textsc{withdrawal\_sent}(id), do(a, s))$
$a = \textsc{withdrawal\_completed}(id) \supset \neg Active(id, do(a, s))$
$a = \textsc{process\_withdrawal\_start}(id) \supset$
$\quad Running(\textsc{process\_withdrawal\_start}(id, do(a, s))$
$a = \textsc{process\_withdrawal\_end}(id) \supset$
$\quad Done(\textsc{process\_withdrawal\_end}(id), do(a, s))$
$a = \textsc{withdrawal\_handled}(id) \supset \neg Active(id, do(a, s))$
$a = \textsc{withdrawal\_handled}(id) \supset Processed(id, do(a, s))$
$a = \textsc{acquire}(id, pool) \supset \neg Waiting(id, pool, do(a, s))$
$a = \textsc{shut\_down} \supset Servers\_stopped(do(a, s))$

Appendix B.1.3. Initial Situation

$$
\begin{aligned}
&Active(id, S_0) = false,\\
&Processed(id, S_0) = false,\\
&Documents\_ok(id, S_0) = false,\\
&Interview(id, S_0) = false,\\
&Job\_offer(id, S_0) = false,\\
&Approval(id, S_0) = false,\\
&Running(\alpha_s, S_0) = false,\\
&Done(\alpha_e, S_0) = false,\\
&Application(id, S_0) = false,\\
&Contract(id, S_0) = false,\\
&Signed\_contract(id, S_0) = false,\\
&Letter\_of\_refusal(id, S_0) = false,\\
&Pool(applicant, S_0) = true,\\
&Pool(company, S_0) = true,\\
&Waiting(id, pool, S_0) = false,\\
&Servers\_stopped(S_0) = false
\end{aligned}
$$

proc $control(bpmn\_process)$
$server\_applicant \parallel server\_company\ \rangle\rangle$
$\langle \neg Stopped \rightarrow true? \rangle$
endProc

proc $server\_applicant$
$(\pi\, id.(\text{ACQUIRE}(id, applicant);\ handle\_applicant(id)))^{\parallel}$
endProc

proc $handle\_applicant(id)$
(**gexec**$(Active(id) \wedge$
$\neg Done(\text{WITHDRAWAL\_BY\_APPLICANT}(id)),$
$applicant\_procedure(id));$
if $(\neg Processed(id) \wedge$
$Done(\text{WITHDRAWAL\_BY\_APPLICANT}(id)))$
then $(\text{CONFIRM\_WITHDRAWAL\_START}(id);$
$Done(\text{CONFIRM\_WITHDRAWAL\_END}(id))?;$
$\text{WITHDRAWAL\_SENT}(id);$
$\text{WITHDRAWAL\_COMPLETED}(id))$
else ()
endIf)
endProc

proc $applicant\_procedure(id)$
$(\text{PREPARE\_APPLICATION\_START}(id);$
$Done(\text{PREPARE\_APPLICATION\_END}(id))?;$
$\text{APPLICATION\_SENT}(id);$
$Done(\text{CONTRACT\_SENT}(id))?;$
$\text{SIGN\_CONTRACT\_START}(id);$
$Done(\text{SIGN\_CONTRACT\_END}(id))?;$
$\text{SIGNED\_CONTRACT\_SENT}(id);$
$\text{COMMUNICATE\_RECRUITMENT\_START}(id);$
$Done(\text{COMMUNICATE\_RECRUITMENT\_END}(id))?;$
$\text{APPLICATION\_FINALISED}(id))\ |$
$(Done(\text{LETTER\_OF\_REFUSAL\_SENT}(id))?;$
$\text{APPLICATION\_FINALISED}(id))$
endProc

```
proc server_company
   (π id.(ACQUIRE(id, company); handle_company(id)))^∥
endProc

proc handle_company(id)
   (gexec((¬Processed(id) ∧
           ¬Done(WITHDRAWAL_SENT(id))),
           company_procedure(id));
   if (¬Processed(id) ∧
      Done(WITHDRAWAL_SENT(id)))
   then (PROCESS_WITHDRAWAL_START(id);
      Done(PROCESS_WITHDRAWAL_END(id))?;
      WITHDRAWAL_HANDLED(id))
   else ()
   endIf )
endProc

proc company_procedure(id)
  (Done(APPLICATION_SENT(id))?;
    CHECK_VALIDITY_START(id);
    Done(CHECK_VALIDITY_END(id));
    if Documents_ok(id)
    then (ORGANIZE_DOCUMENTS_START(id);
      Done(ORGANIZE_DOCUMENTS_END(id))?;
      ASSIGN_CONTACT_PARTNER_START(id) ∥
      VERIFY_PREREQUISITES_START(id);
      (Done(VERIFY_PREREQUISITES_END(id))∧
      Done(ASSIGN_CONTACT_PARTNER_END(id)))?;
      CHECK_APPLICATION_FOR_INTERVIEW_START(id);
      Done(
         CHECK_APPLICATION_FOR_INTERVIEW_END(id))?;
      if Interview(id)
      then
        (PLAN_INTERVIEW_START(id);
        Done(PLAN_INTERVIEW_END(id))?;
        EXECUTE_INTERVIEW_START(id);
        Done(EXECUTE_INTERVIEW_END(id))?;
```

```
        if Job_offer(id)
        then
          (OBTAIN_APPROVAL_START(id);
          Done(OBTAIN_APPROVAL_START(id))?;
          VALIDATE_JOB_OFFER_END(id);
          Done(VALIDATE_JOB_OFFER_END(id))?;
          if Approval(id)
          then
            (PRODUCE_CONTRACT_START(id);
            Done(PRODUCE_CONTRACT_END(id))?;
            CONTRACT_SENT(id);
            Done(SIGNED_CONTRACT_SENT(id))?;
            STORE_SIGNED_CONTRACT_START(id);
            Done(STORE_SIGNED_CONTRACT_END(id))?;
            APPLICATION_ANALYSED(id))
          else
            (PRODUCE_LETTER_OF_REFUSAL_START(id);
            Done(PRODUCE_LETTER_OF_REFUSAL_END(id))?;
            LETTER_OF_REFUSAL_SENT(id);
            APPLICATION_ANALYSED(id))
          endIf )
        else
          (PRODUCE_LETTER_OF_REFUSAL_START(id);
            Done(PRODUCE_LETTER_OF_REFUSAL_END(id))?;
            LETTER_OF_REFUSAL_SENT(id);
            APPLICATION_ANALYSED(id))
        endIf )
      else
        (PRODUCE_LETTER_OF_REFUSAL_START(id);
          Done(PRODUCE_LETTER_OF_REFUSAL_END(id))?;
          LETTER_OF_REFUSAL_SENT(id);
          APPLICATION_ANALYSED(id))
      endIf )
    else
      (PRODUCE_LETTER_OF_REFUSAL_START(id);
        Done(PRODUCE_LETTER_OF_REFUSAL_END(id))?;
        LETTER_OF_REFUSAL_SENT(id);
        APPLICATION_ANALYSED(id))
    endIf )
endProc
```

Appendix C. Case Study Code

The source code for the executable specification of the case study presented in this paper and the instructions to execute it are available at: `https://github.com/angelo-casciani/sitcalc4bpmn/tree/main/pl_models/case_study`. Note that the implementation includes some notational optimizations compared to the formalization described in the paper.

The source code for the software suite, the datasets, and the evaluation results presented in the paper are contained in the following GitHub repository: `https://github.com/angelo-casciani/sitcalc4bpmn`.